ML Notes for AI Engineers

A Complete Beginner-to-Production Reference

---

10 Essential Topics | Explanations + Code | Production-Ready

Math · Python · Core ML · Deep Learning · NLP · Transformers · RAG · Fine-tuning · Prompting · MLOps

[SECTION 1 | PREREQUISITE]

Math Foundations

The minimum math you need — linear algebra, calculus, and probability. You use these every day in ML without explicitly knowing it.

1.1 Linear Algebra — Vectors & Matrices

Every data point in ML is a vector. An image = 1D array of pixels. A sentence = vector of token embeddings. Matrix multiplication IS the forward pass of a neural network.

> *Think of a matrix multiply Wx as 'rotating and scaling' a vector — that's literally what a neural layer does.*

```
import numpy as np
# A data point: 3 features
x = np.array([2.0, 3.0, 1.0])
# Weight matrix (2 neurons, 3 inputs)
W = np.array([[0.1, 0.5, 0.3],
[0.4, 0.2, 0.6]])
# Forward pass = matrix multiply
output = W @ x # shape (2,)
print(output) # [2.0, 2.0]
# Dot product = similarity (used in attention!)
a = np.array([1, 0, 0])
b = np.array([0.9, 0.1, 0])
similarity = np.dot(a, b) # 0.9
```

Key operations to know:

- Dot product — measures similarity between vectors
- Matrix multiply — core of every neural layer forward pass
- Transpose — swap rows and columns (shape: (m,n) -> (n,m))
- SVD (Singular Value Decomposition) — used in PCA, dimensionality reduction

1.2 Calculus — Gradients & Chain Rule

Backpropagation is just the chain rule applied repeatedly. The gradient tells you which direction increases loss — you go the opposite direction to minimize it.

```
import torch
x = torch.tensor([2.0])
w = torch.tensor([0.5], requires_grad=True)
y_true = torch.tensor([1.0])
y_pred = w * x
loss = (y_pred - y_true) ** 2
loss.backward() # compute gradients automatically
print(w.grad) # tensor([2.]) — the gradient
# PyTorch auto-differentiates via autograd engine
```

> ***Tip:*** *You don't need to derive gradients by hand — but understanding that gradient = 'slope' lets you debug exploding/vanishing gradient issues.*

1.3 Probability & Statistics

LLMs output probability distributions over tokens. Loss functions like cross-entropy come directly from probability theory.

```
import numpy as np
# Softmax: converts raw scores (logits) to probabilities
logits = np.array([2.0, 1.0, 0.1])
exp = np.exp(logits - logits.max()) # numerically stable
probs = exp / exp.sum()
print(probs) # [0.659, 0.242, 0.099] — sums to 1.0
# Cross-entropy loss: penalizes wrong distributions
y_true = np.array([1, 0, 0]) # correct class = 0
loss = -np.sum(y_true * np.log(probs + 1e-9))
print(f'loss: {loss:.3f}') # 0.417 — lower = better
```

[SECTION 2 | TOOLING]

Python for ML

NumPy, Pandas, and PyTorch form the entire backbone. Master these three and you can build anything in ML.

2.1 NumPy — Vectorized Operations

Never use Python loops on arrays. NumPy operations run in compiled C code — typically 100x faster. This matters enormously at scale.

```
import numpy as np
# SLOW — Python loop (~200ms for 1M items)
result = [x**2 for x in range(1_000_000)]
# FAST — NumPy vectorized (~2ms)
arr = np.arange(1_000_000)
result = arr ** 2
# Broadcasting: operations across different shapes
X = np.random.randn(100, 10) # 100 samples, 10 features
mean = X.mean(axis=0) # shape (10,)
std = X.std(axis=0)
X_norm = (X - mean) / std # normalize each feature
```

2.2 Pandas — Data Wrangling

Real datasets are messy. Pandas is how you clean, explore, and prepare data before any modeling step.

```
import pandas as pd
import numpy as np
df = pd.read_csv('data.csv')
# Essential EDA
df.info() # column types + null counts
df.describe() # min, max, mean, std, percentiles
df.isnull().sum() # missing values per column
# Clean missing values
df = df.dropna(subset=['target'])
df['age'] = df['age'].fillna(df['age'].median())
# Feature engineering
df['log_income'] = np.log1p(df['income']) # log transform
df = pd.get_dummies(df, columns=['city']) # one-hot encode
```

2.3 PyTorch — Tensor Basics

PyTorch is the de facto standard for AI research and production. Tensors are like NumPy arrays — but they run on GPU and track gradients automatically.

```
import torch
import torch.nn as nn
# Tensor creation
x = torch.randn(32, 768) # batch of 32, embedding dim 768
x = x.cuda() # move to GPU
x = x.cpu().numpy() # back to numpy
# Simple neural net (3 lines!)
model = nn.Sequential(
nn.Linear(768, 256),
nn.ReLU(),
nn.Linear(256, 2)
)
output = model(torch.randn(32, 768))
print(output.shape) # torch.Size([32, 2])
```

[SECTION 3 | FUNDAMENTALS]

Core ML Concepts

The universal concepts that apply to every ML model — from logistic regression to GPT-4. This is the shared vocabulary of the field.

3.1 The ML Pipeline

Every ML project follows the same structure: data -> features -> model -> train -> evaluate -> deploy.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
# 1. Split data FIRST — before any processing
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# 2. Scale features — fit on TRAIN data only!
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # learns mean/std
X_test = scaler.transform(X_test) # applies same scale
# 3. Train
model = LogisticRegression()
model.fit(X_train, y_train)
# 4. Evaluate
print(classification_report(y_test, model.predict(X_test)))
```

> ***Warning:*** *Always fit scalers/encoders on training data only. Fitting on the full dataset is data leakage — your test accuracy becomes a lie.*

3.2 Overfitting vs Underfitting

Overfitting: model memorizes training data, fails on new data. Underfitting: model too simple, fails everywhere. Goal = generalization.

```
# Fix overfitting: Dropout + weight decay
import torch.nn as nn
model = nn.Sequential(
nn.Linear(512, 256),
```

```
nn.Dropout(0.3), # randomly zero 30% of neurons
nn.ReLU(),
nn.Linear(256, 64),
nn.Dropout(0.3),
nn.ReLU(),
nn.Linear(64, 1)
)
optimizer = torch.optim.Adam(
model.parameters(), lr=1e-3, weight_decay=1e-4
)
```

3.3 Loss Functions — Choosing the Right One

```
import torch.nn as nn
# Binary classification (spam / not spam)
loss_fn = nn.BCEWithLogitsLoss()
# Multi-class classification (10 categories)
loss_fn = nn.CrossEntropyLoss()
# Regression (predict price, temperature, etc.)
loss_fn = nn.MSELoss() # mean squared error
loss_fn = nn.HuberLoss() # robust to outliers
```

3.4 Evaluation Metrics

Accuracy lies when classes are imbalanced. Always look at precision, recall, F1, and AUC-ROC.

```
from sklearn.metrics import f1_score, roc_auc_score
# F1 score: balance of precision and recall
f1 = f1_score(y_true, y_pred, average='macro')
auc = roc_auc_score(y_true, y_proba)
# Regression metrics
from sklearn.metrics import mean_absolute_error, r2_score
mae = mean_absolute_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred) # 1.0 = perfect
```

[SECTION 4 | CORE AI]

Deep Learning

Neural networks from scratch. Understanding these internals makes transformers and LLMs demystified — they are just stacked, attention-enhanced neural nets.

4.1 The Training Loop

Every deep learning model trains using the same 5-step loop. Memorize this — it is the heartbeat of all ML.

```
model = MyModel()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
loss_fn = nn.CrossEntropyLoss()
for epoch in range(10):
for X_batch, y_batch in dataloader:
# 1. Forward pass
logits = model(X_batch)
# 2. Compute loss
loss = loss_fn(logits, y_batch)
# 3. Zero old gradients (CRITICAL — don't skip!)
optimizer.zero_grad()
# 4. Backward pass — compute gradients
loss.backward()
# 5. Update weights
optimizer.step()
print(f'Epoch {epoch}: loss={loss.item():.4f}')
```

4.2 Activation Functions

Without activations, stacking layers collapses to one linear transform. Activations add non-linearity — the power to learn complex patterns.

```
import torch.nn.functional as F
# ReLU: f(x) = max(0, x) — fastest, most common
# Use in: hidden layers of most networks
# GELU: smoother ReLU — standard in transformers (GPT, BERT)
# Slightly better for language models in practice
x = torch.randn(4)
print(F.relu(x)) # zeros out negatives
```

```
print(F.gelu(x)) # smooth, keeps small negatives
print(F.softmax(x, dim=-1)) # sums to 1.0
```

4.3 Batch Norm & Layer Norm

Without normalization, activations drift to huge or tiny values during training. BatchNorm = CNNs. LayerNorm = Transformers.

```
# BatchNorm — normalizes across batch dimension
nn.BatchNorm1d(256) # 256 features
# LayerNorm — normalizes across feature dimension
# Standard in GPT, BERT, all modern transformers
nn.LayerNorm(768) # 768-dim embeddings
# Standard transformer block pattern:
class TransformerBlock(nn.Module):
def forward(self, x):
x = x + self.attn(self.norm1(x)) # residual
x = x + self.ffn(self.norm2(x))
return x
```

4.4 Optimizers — AdamW

Use AdamW for virtually everything in 2024+. It adapts learning rates per parameter and properly decouples weight decay from gradient update.

```
# AdamW — standard for LLMs and transformers
optimizer = torch.optim.AdamW(
model.parameters(),
lr=3e-4, # Karpathy's constant — start here
weight_decay=0.01 # L2 regularization
)
# Cosine decay learning rate schedule
from torch.optim.lr_scheduler import CosineAnnealingLR
scheduler = CosineAnnealingLR(optimizer, T_max=100)
for epoch in range(100):
train_one_epoch()
scheduler.step()
```

> ***Tip:*** *Learning rate is the most impactful hyperparameter. Too high = diverge. Too low = very slow. Start with 3e-4 and adjust based on loss curves.*

[SECTION 5 | LANGUAGE]

NLP Fundamentals

How text becomes numbers — the indispensable bridge between language and the math that models understand.

5.1 Tokenization

Models cannot read words — they read token IDs. Tokenizers split text into subword chunks using BPE (Byte Pair Encoding) and map each chunk to an integer.

> *1 token = ~0.75 words. 'ChatGPT' = 1 token. 'supercalifragilistic' = 5 tokens. LLM pricing is per token — always count tokens, not characters.*

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained('gpt2')
text = 'Transformers are amazing!'
tokens = tokenizer(text, return_tensors='pt')
print(tokens['input_ids'])
# tensor([[41962, 364, 389, 4998, 0]])
print(tokenizer.convert_ids_to_tokens(tokens['input_ids'][0]))
# ['Transform', 'ers', ' are', ' amazing', '!']
# Always count tokens for cost estimation
n_tokens = tokens['input_ids'].shape[-1]
print(f'Token count: {n_tokens}') # 5 tokens
```

5.2 Word Embeddings

Embeddings map tokens to dense vectors where meaning is encoded as geometry. Similar words are close in vector space — this is how models understand semantics.

```
from transformers import AutoModel, AutoTokenizer
import torch
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModel.from_pretrained('bert-base-uncased')
def get_embedding(text):
inputs = tokenizer(text, return_tensors='pt')
with torch.no_grad():
outputs = model(**inputs)
return outputs.last_hidden_state.mean(dim=1)
```

```
cos = torch.nn.CosineSimilarity(dim=-1)
print(cos(get_embedding('cat'), get_embedding('dog'))) # ~0.92
print(cos(get_embedding('cat'), get_embedding('car'))) # ~0.61
```

5.3 HuggingFace Pipelines

The fastest way to run NLP tasks — use a pretrained model via the pipeline API. Zero boilerplate, production quality.

```
from transformers import pipeline

# Sentiment analysis
clf = pipeline('sentiment-analysis')
print(clf('This product is great!'))
# [{'label': 'POSITIVE', 'score': 0.9998}]

# Named Entity Recognition
ner = pipeline('ner', aggregation_strategy='simple')
print(ner('Elon Musk founded Tesla in Palo Alto.'))
# [{'entity': 'PER', 'word': 'Elon Musk'},
# {'entity': 'ORG', 'word': 'Tesla'},
# {'entity': 'LOC', 'word': 'Palo Alto'}]
```

[SECTION 6 | CRITICAL]

Transformers & Attention

The architecture behind every major AI model — GPT-4, Claude, Gemini, BERT. Non-negotiable knowledge for any AI engineer.

6.1 Self-Attention — The Core Mechanism

Attention lets each token look at all other tokens and decide which are relevant. This is how 'bank' knows it means 'financial institution' vs 'river bank' — from context alone.

> *Q = 'What am I looking for?' K = 'What do I contain?' V = 'What should I return?' — Find relevant keys for each query, return their values.*

```
import torch, math
def scaled_dot_product_attention(Q, K, V, mask=None):
d_k = Q.shape[-1]
# Attention scores: Q @ K^T / sqrt(d_k)
scores = torch.matmul(Q, K.transpose(-2, -1))
scores = scores / math.sqrt(d_k)
# Optional causal mask: decoder can't see future tokens
if mask is not None:
scores = scores.masked_fill(mask == 0, -1e9)
attn_weights = torch.softmax(scores, dim=-1)
output = torch.matmul(attn_weights, V)
return output, attn_weights
```

6.2 Multi-Head Attention

Run attention h times in parallel with different learned projections. Each head learns different relationship types — syntax, coreference, semantics.

```
class MultiHeadAttention(nn.Module):
def __init__(self, d_model=512, n_heads=8):
super().__init__()
self.d_k, self.n_heads = d_model // n_heads, n_heads
self.W_q = nn.Linear(d_model, d_model)
self.W_k = nn.Linear(d_model, d_model)
self.W_v = nn.Linear(d_model, d_model)
self.W_o = nn.Linear(d_model, d_model)
```

```
def forward(self, x):
B, T, C = x.shape
def proj(W): return W(x).reshape(B,T,self.n_heads,self.d_k).transpose(1,2)
Q, K, V = proj(self.W_q), proj(self.W_k), proj(self.W_v)
out, _ = scaled_dot_product_attention(Q, K, V)
return self.W_o(out.transpose(1,2).reshape(B, T, C))
```

6.3 BERT vs GPT

The most important architectural distinction: BERT-style = understand text. GPT-style = generate text.

```
# BERT: encoder-only, BIDIRECTIONAL attention
# Sees ALL tokens at once — good for classification, NER
# GPT: decoder-only, CAUSAL (masked) attention
# Only sees past tokens — good for generation, chat
from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained('gpt2')
tokenizer = AutoTokenizer.from_pretrained('gpt2')
inputs = tokenizer('The capital of France is', return_tensors='pt')
outputs = model.generate(**inputs, max_new_tokens=5)
print(tokenizer.decode(outputs[0]))
# 'The capital of France is Paris'
```

[SECTION 7 | PRODUCTION AI]

RAG Systems

Retrieval-Augmented Generation — how to give LLMs access to your private knowledge base. This is what most AI Engineer jobs actually build day-to-day.

7.1 Why RAG?

LLMs have a training cutoff and can't access your company's internal docs. RAG: retrieve relevant text chunks from your data, inject them into the prompt, let the LLM reason over them.

> *Use RAG when data changes frequently, you need citations, or the knowledge base is large. Use fine-tuning when you need a specific output style or latency is critical.*

7.2 Building a RAG Pipeline

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.chains import RetrievalQA
# Step 1: Chunk documents
splitter = RecursiveCharacterTextSplitter(
chunk_size=500, chunk_overlap=50
)
chunks = splitter.split_documents(documents)
# Step 2: Embed chunks into vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)
# Step 3: Build retriever + LLM chain
retriever = vectorstore.as_retriever(
search_type='mmr', search_kwargs={'k': 4}
)
qa_chain = RetrievalQA.from_chain_type(
llm=ChatOpenAI(model='gpt-4o'), retriever=retriever
)
# Step 4: Query
answer = qa_chain.invoke('What was Q3 revenue?')
print(answer['result'])
```

7.3 Vector Databases

Store millions of embeddings and find nearest neighbors in milliseconds. Core infrastructure for any RAG or semantic search system.

```
import faiss, numpy as np
dim = 1536 # OpenAI ada-002 dimension
index = faiss.IndexFlatIP(dim) # inner product = cosine sim
index.add(np.array(embeddings, dtype=np.float32))
query = np.array([query_embedding], dtype=np.float32)
distances, indices = index.search(query, k=5)
# Production vector DB options:
# Pinecone — managed, scales to billions
# Qdrant — open source, rich filtering
# Weaviate — multi-modal, GraphQL API
# ChromaDB — local dev, very easy setup
```

> ***Tip:*** *Chunking strategy matters more than model choice. 500-token chunks with 50-token overlap is a solid default. Too large = noisy retrieval. Too small = loses context.*

[SECTION 8 | ADVANCED]

Fine-tuning LLMs

Adapt a pretrained model to your specific domain or task — without training from scratch (which costs millions of dollars).

8.1 LoRA — Low-Rank Adaptation

Full fine-tuning updates billions of weights. LoRA freezes the original model and adds tiny trainable adapter matrices — same quality, 100x fewer trainable parameters.

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM
# Load model in 4-bit (QLoRA — quantized LoRA)
model = AutoModelForCausalLM.from_pretrained(
'mistralai/Mistral-7B-v0.1',
load_in_4bit=True # 28GB -> 4GB
)
lora_config = LoraConfig(
r=16, # rank
lora_alpha=32, # scaling
target_modules=['q_proj', 'v_proj'],
lora_dropout=0.1,
task_type='CAUSAL_LM'
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable%: 0.058 — training <0.1% of parameters!
```

8.2 Training with SFTTrainer

```
from transformers import TrainingArguments
from trl import SFTTrainer
args = TrainingArguments(
output_dir='./output',
num_train_epochs=3,
per_device_train_batch_size=4,
gradient_accumulation_steps=4, # effective batch = 16
learning_rate=2e-4,
fp16=True, # mixed precision
```

```
)
trainer = SFTTrainer(
model=model, args=args,
train_dataset=train_dataset,
dataset_text_field='text',
max_seq_length=2048,
)
trainer.train()
model.save_pretrained('./my-finetuned-model')
```

> *Dataset format for chat models — use ChatML: <|im_start|>user Your message<|im_end|> <|im_start|>assistant Response<|im_end|>*

[SECTION 9 | AI ENGINEERING]

Prompt Engineering

The fastest lever you have. Good prompts can outperform fine-tuning for most tasks — often underestimated by engineers.

9.1 Core Techniques

```
# Zero-shot: just ask, no examples
prompt = "Classify the sentiment: 'The food was cold.'"

# Few-shot: show examples to guide output format
prompt = """Classify as POSITIVE/NEGATIVE/NEUTRAL.
Input: 'Great service!' -> POSITIVE
Input: 'Too expensive.' -> NEGATIVE
Input: 'The food was cold.' ->"""

# Chain-of-Thought: ask to reason step by step
# Dramatically improves math, logic, reasoning
prompt = """Solve step by step:
A store sells 3 shirts for $45.
How much do 7 shirts cost?
Let me think:"""
```

9.2 System Prompts & Structured Output

```
import anthropic, json

client = anthropic.Anthropic()

message = client.messages.create(
model='claude-sonnet-4-5',
max_tokens=1024,
system="""You are a data extraction assistant.
Always respond with valid JSON only.
Schema: {name: str, age: int, city: str}""",
messages=[{
'role': 'user',
'content': 'Extract: John is 28 years old from Mumbai.'
}]
)

data = json.loads(message.content[0].text)
# {'name': 'John', 'age': 28, 'city': 'Mumbai'}
```

9.3 Tool Use / Function Calling

Give the LLM tools it can decide to invoke. This is how AI agents work — model reasons about what to do, then calls the right function.

```
tools = [{
'name': 'get_weather',
'description': 'Get current weather for a city',
'input_schema': {
'type': 'object',
'properties': {'city': {'type': 'string'}},
'required': ['city']
}
}]
response = client.messages.create(
model='claude-sonnet-4-5', max_tokens=1024,
tools=tools,
messages=[{'role':'user', 'content':'Weather in Delhi?'}]
)
# Returns: tool_use block -> city='Delhi'
# Your code runs the function, sends result back
# Model reads result and writes final answer
```

[SECTION 10 | PRODUCTION]

MLOps & Deployment

A model that isn't deployed is a Jupyter notebook. MLOps is how you take ML from experiment to reliable production.

10.1 Model Serving with FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import torch
app = FastAPI()
model = torch.load('model.pt')
model.eval()
class PredictRequest(BaseModel): text: str
class PredictResponse(BaseModel): label: str; confidence: float
@app.post('/predict', response_model=PredictResponse)
async def predict(req: PredictRequest):
with torch.no_grad():
tokens = tokenizer(req.text, return_tensors='pt')
logits = model(**tokens).logits
probs = torch.softmax(logits, dim=-1)
label = id2label[probs.argmax().item()]
conf = probs.max().item()
return PredictResponse(label=label, confidence=conf)
# uvicorn main:app --host 0.0.0.0 --port 8000
```

10.2 Experiment Tracking with MLflow

```
import mlflow
mlflow.set_experiment('sentiment-classifier')
with mlflow.start_run():
mlflow.log_param('learning_rate', 3e-4)
mlflow.log_param('batch_size', 32)
train()
mlflow.log_metric('val_f1', 0.923)
mlflow.log_metric('val_loss', 0.187)
mlflow.pytorch.log_model(model, 'model')
```

```
# Compare all runs: mlflow ui -> localhost:5000
```

10.3 Quantization — Ship Faster & Cheaper

A 7B param model at FP32 = 28GB VRAM. At INT4 = 4GB. Quantization compresses weights with minimal quality loss.

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
import torch
bnb_config = BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_compute_dtype=torch.bfloat16,
bnb_4bit_use_double_quant=True,
bnb_4bit_quant_type='nf4'
)
model = AutoModelForCausalLM.from_pretrained(
'mistralai/Mistral-7B-v0.1',
quantization_config=bnb_config,
device_map='auto'
)
# 7B model now fits on a single 8GB GPU!
```

10.4 Docker — Reproducible Environments

```
# Dockerfile for ML inference service
FROM pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt --no-cache-dir
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
# requirements.txt
# torch==2.1.0 transformers==4.36.0 fastapi==0.104.1
# uvicorn==0.24.0 peft==0.6.2 bitsandbytes==0.41.3
```

> ***Tip:*** *For production inference: llama.cpp (CPU), vLLM (GPU high-throughput), or Ollama (local dev). Never serve models from Jupyter notebooks.*

End of ML Notes for AI Engineers

Math · Python · ML · Deep Learning · NLP · Transformers · RAG · Fine-tuning · Prompting · MLOps